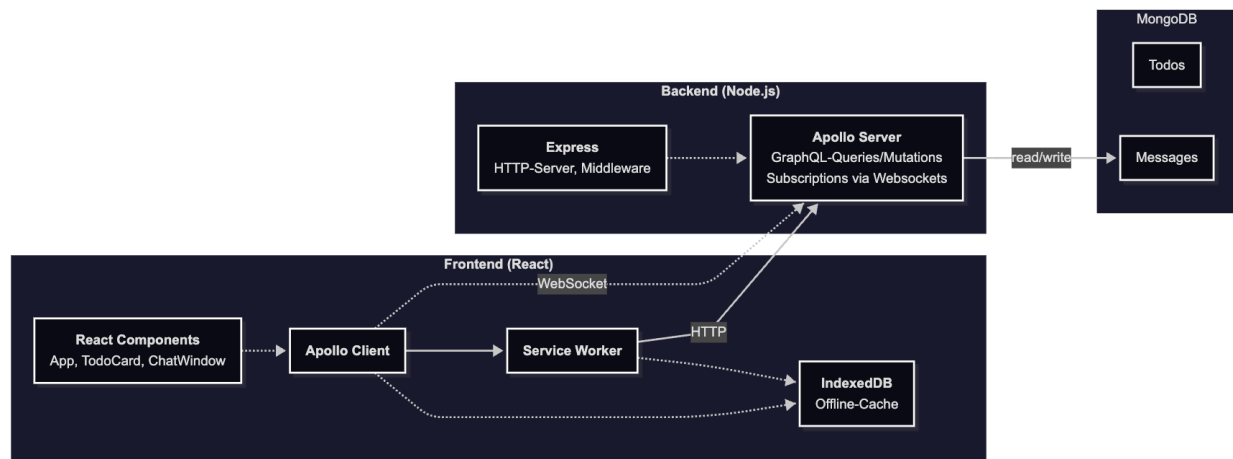


# Webtech – Abgabe 3 – MEAN

---

## Aufgabe 4

### Architektur



### Rolle von WebSockets

Für die Chat-Funktion der Anwendung werden WebSockets eingesetzt. Die Verbindung wird über den Endpunkt `ws://localhost:4000/chat` aufgebaut und ermöglicht eine bidirektionale Echtzeitkommunikation zwischen Client und Server.

Die Chat-Architektur ist raumbasiert aufgebaut. Jeder Todo-Eintrag besitzt einen eigenen Chat-Raum, der über die jeweilige Todo-ID als Query-Parameter identifiziert wird. Dadurch werden Nachrichten ausschließlich an die Benutzer verteilt, die dem entsprechenden Todo-Chat zugeordnet sind.

Beim Verbindungsaufbau sendet der Server eine Nachricht vom Typ `HISTORY`, welche die bisher gespeicherten Chat-Nachrichten enthält. Neue Nachrichten werden als `NEW_MESSAGE` an alle verbundenen Clients desselben Chat-Raums übertragen.

Falls keine Verbindung zum WebSocket-Server hergestellt werden kann, werden bereits gespeicherte Nachrichten aus der lokalen IndexedDB geladen. Zusätzlich wird der aktuelle

---

Verbindungsstatus überwacht und zwischen geöffneten (OPEN) und geschlossenen (CLOSED) Verbindungen unterschieden.

### **Aufgaben der WebSockets:**

- Echtzeit-Kommunikation zwischen Clients
- Separate Chat-Räume pro Todo-ID
- Übertragung von Nachrichtenhistorien (HISTORY)
- Verteilung neuer Nachrichten in Echtzeit (NEW\_MESSAGE)
- Offline-Fallback über IndexedDB
- Überwachung des Verbindungsstatus

### **Eingesetzte PWA-Technologien**

Die Anwendung wurde als Progressive Web App (PWA) umgesetzt, um Offline-Funktionalität und eine installierbare Benutzeroberfläche bereitzustellen.

Ein Service Worker übernimmt die Installation und Aktivierung der Anwendung sowie die Verwaltung verschiedener Cache-Strategien. Beim Aktivieren werden veraltete Cache-Versionen automatisch entfernt.

Die Datei manifest.json definiert die Metadaten der Anwendung, darunter Anwendungsname, Icons, Theme-Farbe und die Ausführung im Standalone-Modus.

Für die Zwischenspeicherung werden unterschiedliche Strategien verwendet:

- Cache-First für die App-Shell (index.html, manifest.json)
- Network-First für JavaScript-, CSS- und Bilddateien
- Offline-Fallback auf bereits gecachte Ressourcen bei Netzwerkfehlern

WebSocket-Verbindungen (ws:// bzw. wss://) werden bewusst von der Cache-Verarbeitung ausgeschlossen, da Echtzeitverbindungen nicht sinnvoll zwischengespeichert werden können.

---

## Eingesetzte PWA-Technologien:

- Service Worker
- Web App Manifest
- App-Shell-Caching
- Offline-Unterstützung
- Automatische Bereinigung veralteter Caches
- Installierbare Anwendung (Standalone-Modus)

## Verwendete Browser-Persistenz (IndexedDB)

Für die lokale Datenspeicherung wird IndexedDB verwendet. Die Anwendung legt die Datenbank todo-app-db in Version 1 an.

Die Datenbank enthält zwei Object Stores:

- todos für die lokale Speicherung von Todo-Daten
- messages für Chat-Nachrichten mit zusätzlichem Index auf todold

Über die Funktionen saveTodos(), loadTodos(), saveMessages() und loadMessages() können Daten lokal gespeichert und wieder geladen werden.

Die Persistenzschicht dient sowohl der Offline-Verfügbarkeit als auch der Reduzierung von Netzwerkanfragen. Dadurch bleiben Todos und Chat-Nachrichten auch bei fehlender Internetverbindung verfügbar.

Verwendungszwecke:

- Offline-Zugriff auf Todos
- Offline-Zugriff auf Chat-Nachrichten
- Lokales Caching von Anwendungsdaten
- Schnellere Wiederherstellung des Anwendungszustands

---

## Beschreibung der entwickelten Web Components

### ToDoList

Die Komponente ToDoList stellt die zentrale Verwaltungsansicht der Anwendung dar und integriert die gesamte GraphQL-Kommunikation.

Folgende GraphQL-Queries werden verwendet:

- GET\_TODOS
- TODO\_DETAIL

Zusätzlich unterstützt die Komponente verschiedene Filtermöglichkeiten nach Status, Tags und Priorität.

Für Änderungen an den Daten werden folgende Mutations verwendet:

- CREATE\_TODO
- UPDATE\_TODO
- DELETE\_TODO

Darüber hinaus werden weitere Funktionen über folgende Mutations bereitgestellt:

- ADD\_COMMENT
- ADD\_CHECKLIST\_ITEM

### ToDoCard

Die Komponente ToDoCard dient der Darstellung einzelner Todos in Kartenform.

Angezeigt werden unter anderem:

- Titel des Todos
- Aktueller Status
- Priorität (optional)

Der Status wird über farblich gekennzeichnete Span-Elemente visualisiert. Zusätzlich besitzt jede Karte einen Button zum Öffnen des zugehörigen Chats.

---

## ChatWindow

Die Komponente ChatWindow stellt die Chat-Funktion eines Todos bereit.

Sie verwaltet die WebSocket-Verbindung, zeigt den Nachrichtenverlauf an und informiert den Benutzer über den aktuellen Verbindungsstatus (isConnected). Neue Nachrichten werden automatisch in der IndexedDB gespeichert, um eine Offline-Verfügbarkeit sicherzustellen.

Beim Entfernen der Komponente aus dem DOM wird die Verbindung ordnungsgemäß geschlossen und alle zugehörigen Ressourcen werden freigegeben (Graceful Cleanup).

Funktionen der Komponente:

- Anzeige des Nachrichtenverlaufs
- Anzeige des Nachrichtentexts
- Echtzeit-Kommunikation über WebSockets
- Speicherung neuer Nachrichten in IndexedDB
- Anzeige des Verbindungsstatus
- Automatisches Cleanup beim Unmount